

Fecha: 27 de mayo de 2026

Integrante:

Juan Felipe Quiroga Medina

Curso:

Sistemas Operativos



UNIVERSIDAD
NACIONAL
DE COLOMBIA

Taller Sincronización en Go

Para el taller se utilizarán las Goroutines por medio de Canales y WorkersPool. Esto permite controlar el número de Hilos que se crean en la concurrencia, tener una mayor organización de tareas a realizar por los hilos y un mejor manejo de recursos de la máquina.

1. Explicación de código

1.1. Funciones

```
11 func puntosDentroDelCirculo(puntoX float64, puntoY float64) float64 {
12     return math.Sqrt((math.Pow(puntoX,2.0) + math.Pow(puntoY,2.0)))
13 }
14
15 func monteCarlo(puntos_circulo float64, lista_len float64) float64{
16     var pi = 4 * (puntos_circulo/lista_len)
17     return pi
18 }
19
```

Figura 1: Funciones del programa

Se definen dos funciones: **puntosDentroDelCirculo**, encargada de calcular la distancia del punto (x, y) dado, realiza la fórmula de distancia entre dos puntos:

$$distancia = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

Tener en cuenta que $(x_1, y_1) = (0, 0)$, que equivale al centro del círculo en el plano. Para la función **Monte Carlo** se realiza el cálculo de Pi por la siguiente fórmula:

$$\pi = 4 \times \frac{\text{puntos dentro de círculo}}{\text{total de puntos}}$$

1.2. Variables

```
22     size := 100000000
23     workerCount := runtime.NumCPU()
24     batchSize := 10000
25     var pi_monte_carlo float64
26     jobs := make(chan int, workerCount)
27     resultado := make(chan int, workerCount)
28     var wg sync.WaitGroup
29
```

Figura 2: Variables

Definimos las variables del programa: **size** es el tamaño de puntos que se van a calcular para determinar el tamaño de π ; **workerCount** da el número de CPUs de la máquina que se pueden utilizar para ejecutar las tareas en paralelo; **batchSize** es para indicar el tamaño de subtareas a realizar por una de las Goroutines, en este caso, obtener un punto (x, y) en el plano y calcular su distancia; **pi_monte_carlo** es donde guardaremos el cálculo de π ; **jobs** y **resultado** son los canales de tamaño workerCount; **wg** se utiliza para administrar las Goroutines.

1.3. Canales

Las tuberías que conectan las Goroutines administrarán las tareas a realizar y almacenarán los resultados de cada tarea.

1.4. Administración de trabajadores

```
30     for w := 0; w < workerCount; w++ {
31         wg.Add(1)
32         go func(id int) {
33             defer wg.Done()
34             puntosInternos := 0
35             for batch := range jobs {
36                 for i := 0; i < batch; i++ {
37                     x := puntosDentroDelCirculo(rand.Float64() * 2 - 1, rand.Float64() * 2 - 1)
38                     if x <= 1 {
39                         puntosInternos++
40                     }
41                 }
42             }
43             resultado <- puntosInternos
44         }(w)
45     }
```

Figura 3: Administración de trabajadores

Se realiza un ciclo for para activar a los trabajadores; inicialmente no hay trabajos creados, por lo que quedan en espera hasta que se manden tareas. **wg.Add(1)** se usa para indicar cuántas llamadas de finalización debe esperar, que en este caso sería una. La instrucción **defer wg.Done()** indicaría la finalización de la Goroutine y esta se ejecutará al final de la go func(). En la línea 36, cada trabajador realizará la tarea por medio de un ciclo for; en la variable **puntosInternos** se acumulará el número de puntos dentro del círculo y en la línea 43 insertamos el valor en el canal de resultados.

1.5. Administración de trabajos

```

47     puntosFaltantes := size
48     for puntosFaltantes > 0 {
49         b := batchSize
50         if puntosFaltantes < b {
51             b = puntosFaltantes
52         }
53         jobs <- b
54         puntosFaltantes -= b
55     }
56     close(jobs)
57

```

Figura 4: Administración de trabajos

Las tareas se determinan por las variables **puntosFaltantes** y **batchSize**. Como cada tarea consiste en realizar el cálculo de la distancia de un punto generado aleatoriamente y determinar si está dentro del círculo, realizado n veces, siendo $n = \text{batchSize}$, el número de tareas creadas sería:

$$\text{jobs} = \left\lfloor \frac{\text{size}}{\text{batchsize}} \right\rfloor + 1$$

Por cada trabajo generado, el número total de puntos a calcular disminuye en el tamaño de **batchSize**. Al mandar las tareas al canal **jobs**, son ejecutadas inmediatamente. En la línea 56 se cierra el canal indicando que no se enviarán más tareas.

1.6. Finalización

```

59     go func() {
60         wg.Wait()
61         close(resultado)
62     }()
63
64     fmt.Println("resultado:", resultado)
65     totalInternos := 0
66     for v := range resultado {
67         totalInternos += v
68     }
69
70     pi_monte_carlo = monteCarlo(float64(totalInternos), float64(size))
71     fmt.Println("puntos dentro:", totalInternos)
72     fmt.Println("total puntos:", size)
73     fmt.Println("pi =", pi_monte_carlo)
74

```

Figura 5: Finalización programa

En la línea 59 se ejecuta la `go func()` que se encarga de esperar la finalización de las Goroutines con `wg.Wait()` y cerrar el canal de resultados. El total de puntos internos se determina en la línea 66, y en la línea 70 se obtiene la aproximación de π con la función **monteCarlo** y se imprimen los resultados.

1.7. Comparación tiempo de ejecución

Por medio del comando **time** se compara el tiempo de ejecución del programa en paralelo y serial. Se ejecutará el programa tres veces y se calcula el promedio.

N puntos	paralelo	serial
100000	5,154 ms	9,032 ms
1000000	41,569 ms	88,411 ms
10000000	371,1 ms	799,65 ms
100000000	3,389 s	7,872 s

Cuadro 1: Tiempo de ejecución

```

● @Cata0103 → /workspaces/JAVA/S0 (main) $ go run taller.go
puntos dentro: 78540725
total puntos: 100000000
pi = 3.141629
Tiempo de ejecución: 3.52980221s
● @Cata0103 → /workspaces/JAVA/S0 (main) $ go run taller.go
puntos dentro: 78537027
total puntos: 100000000
pi = 3.14148108
Tiempo de ejecución: 3.434639868s
● @Cata0103 → /workspaces/JAVA/S0 (main) $ go run taller.go
puntos dentro: 78536055
total puntos: 100000000
pi = 3.1414422
Tiempo de ejecución: 3.830658861s
❖ @Cata0103 → /workspaces/JAVA/S0 (main) $

```

Figura 6: Ejecución paralela

```

● @Cata0103 → /workspaces/JAVA/S0 (main) $ go run taller_serial.go
puntos dentro: 78541910
total puntos: 100000000
pi = 3.1416764
Tiempo de ejecución: 7.880465817s
● @Cata0103 → /workspaces/JAVA/S0 (main) $ go run taller_serial.go
puntos dentro: 78528129
total puntos: 100000000
pi = 3.14112516
Tiempo de ejecución: 7.942147555s
● @Cata0103 → /workspaces/JAVA/S0 (main) $ go run taller_serial.go
puntos dentro: 78532053
total puntos: 100000000
pi = 3.14128212
Tiempo de ejecución: 7.961150538s
❖ @Cata0103 → /workspaces/JAVA/S0 (main) $

```

Figura 7: Ejecución serial